

ENTERPRISE PLATFORM

High-Level Design Document

HLD — PART 3 OF 3

Scalability | Cost Model | Module Integration | Evolution Roadmap

Document Scope

This is Part 3 of the 3-part HLD. Parts 1 and 2 covered architecture decisions, system topology, component design, data architecture, multi-tenancy, security, event-driven messaging, caching, and observability. This final part covers the scalability and performance strategy, the cost model per phase, integration contracts for every future module, the complete evolution roadmap from Modular Monolith to Microservices, and team and process recommendations.

16. Scalability Strategy

Scalability in this architecture is not an emergency measure triggered when things go wrong. It is a planned progression defined before any traffic arrives. Each scaling action is triggered by a measurable condition, not by guesswork.

16.1 Horizontal vs Vertical Scaling Decision

Component	Scaling Strategy	Reason
FastAPI Backend	Horizontal — add more containers	Stateless. Any pod serves any request. Linear scaling.
Next.js Frontend	Horizontal — Vercel edge network	Stateless SSR. Vercel auto-scales globally.
PostgreSQL	Vertical first, then read replicas	Stateful. Vertical is simpler. Read replicas for reporting.
Redis	Vertical first, then cluster mode	Mostly ephemeral. Cluster when memory exceeds 10GB.
Nginx / API Gateway	Horizontal — multiple instances behind LB	Stateless. Config-driven. Easy to replicate.
Background Workers	Horizontal — add more consumer instances	Event consumers are stateless and idempotent.

16.2 Why FastAPI Scales Horizontally Without Code Changes

The FastAPI backend is stateless by design. This is not an accident — it was an explicit architectural requirement enforced from Module 1.

Design Decision	Why It Enables Horizontal Scaling
JWT tokens in HttpOnly cookies	Token validation is stateless — any pod can validate any token using the shared SECRET_KEY
Sessions stored in PostgreSQL	No in-memory session state. Any pod can serve a refresh request.
OTPs stored in Redis	Shared Redis is visible to all pods. No local state.
Rate limits in Redis	Sliding window counter is shared. All pods contribute to the same counter.
No background threads in the app	Background tasks use Redis Streams. Workers are separate processes.
Config via environment variables	Each pod reads its own environment. No shared config state.

16.3 Scaling Triggers — When to Scale Each Component

Component	Scale Trigger	Scaling Action
FastAPI pods	CPU > 60% sustained for 3 min	Add 2 pods. HPA handles automatically in Phase 3.
FastAPI pods	p95 latency > 500ms	Add pods. Investigate slow queries in parallel.
PostgreSQL	CPU > 70% OR query time > 100ms	Upgrade instance size. Add read replica for reports.
PostgreSQL connections	PgBouncer queue depth > 10	Increase pool size or upgrade instance.
Redis	Memory > 70% of limit	Upgrade instance. Enable cluster mode if > 10GB.
Redis	Hit rate < 80%	Review cache TTLs. Investigate cache stampede patterns.
Event queue	Consumer lag > 10,000 messages	Add consumer instances for the lagging stream.

16.4 Database Query Performance Strategy

Database performance degrades predictably as data grows. These strategies prevent degradation before it happens:

- Indexes created before the table has significant data — cheaper and faster than adding later
- Composite indexes match exact query patterns — not just individual columns
- Partial indexes on filtered queries — `idx_sessions_user_valid` only indexes active sessions
- EXPLAIN ANALYZE run on every new complex query before it reaches production
- Slow query log enabled — any query over 100ms is flagged for review
- Table partitioning for `audit_logs` when rows exceed 10 million — partition by month
- Archival policy — audit logs older than 2 years moved to cold storage (S3 Glacier)

17. Cost Model — Every Phase

Costs are modelled honestly. Every line item is a real service with a real price. The goal is to show that the architecture is cost-efficient at every stage and that the transition between phases is financially planned — not reactive.

17.1 Phase 1 — MVP Infrastructure (~\$65/month)

Service	Provider	Monthly Cost	Notes
VPS Server (4 vCPU, 8GB RAM)	Hetzner CX31	\$18	All services on one machine
CDN + DDoS + WAF	Cloudflare Free	\$0	Covers most attack vectors
SSL Certificate	Let's Encrypt	\$0	Auto-renewing via Certbot
Domain Name	Any registrar	\$1	Amortised annually
Twilio SMS	Twilio	\$0-\$15	First 100 SMS free. \$0.0079/SMS after.
SendGrid Email	SendGrid Free	\$0	100 emails/day free
Sentry Error Tracking	Sentry Free	\$0	5,000 errors/month free
Uptime Monitoring	UptimeRobot Free	\$0	5-minute check interval
Backup Storage (pg_dump)	Cloudflare R2	\$1	First 10GB free
CI/CD Pipeline	GitHub Actions Free	\$0	2,000 minutes/month free
TOTAL		~\$35-65/month	Scales to 10,000 DAU comfortably

17.2 Phase 2 — Growth Infrastructure (~\$450/month)

Service	Provider	Monthly Cost	Notes
ECS Fargate (2-8 tasks)	AWS	\$60-\$180	0.25 vCPU / 0.5GB per task
RDS PostgreSQL db.t4g.medium	AWS	\$60	Multi-AZ adds \$60 more
ElastiCache Redis t4g.micro	AWS	\$15	Single node
Application Load Balancer	AWS	\$20	Plus \$0.008/LCU
Next.js on Vercel Pro	Vercel	\$20	Global edge, 1TB bandwidth

Service	Provider	Monthly Cost	Notes
Cloudflare Pro	Cloudflare	\$20	Advanced WAF rules
S3 Object Storage	AWS	\$5-\$20	User uploads, assets
SendGrid Essentials	SendGrid	\$20	50,000 emails/month
Twilio SMS	Twilio	\$20-\$50	Volume dependent
Sentry Team	Sentry	\$26	50,000 errors/month
AWS Secrets Manager	AWS	\$5	~10 secrets
TOTAL		~\$300-500/month	Scales to 500,000 DAU

17.3 Phase 3 — Enterprise Infrastructure (~\$3,500/month base)

Service	Provider	Monthly Cost	Notes
EKS Cluster (3 nodes)	AWS	\$300	Node group: m5.xlarge x3
EKS Pods (HPA 3-50)	AWS	\$200-\$800	Scales with traffic
Aurora PostgreSQL Serverless v2	AWS	\$400-\$800	Scales with query load
ElastiCache Redis Cluster	AWS	\$200	3 shards, 6 nodes
MSK Kafka (3 brokers)	AWS	\$400	kafka.m5.large
Global Load Balancer	AWS	\$100	Multi-region routing
Vercel Enterprise	Vercel	\$400	Enterprise SLA, custom domains
Cloudflare Enterprise	Cloudflare	\$500	Advanced bot management
Sentry Business	Sentry	\$80	Performance monitoring
HashiCorp Vault	HCP	\$100	Secrets management
TOTAL		~\$3,000-5,000/month	Scales to 5M+ DAU

Revenue Context

At 500,000 users on a SaaS model at even \$1/user/month average revenue, the platform generates \$500,000/month. Phase 2 infrastructure at \$450/month represents 0.09% of revenue. The architecture is cost-efficient at every phase.

18. Module Integration Contracts

Each future module has a defined integration contract with Module 1 (Auth). This contract specifies exactly what Module 1 provides, what the new module must implement, and what events connect them. These contracts are defined now so Module 1 never needs to be redesigned when new modules are added.

18.1 Module 2 — User Profiles and Settings

Contract Item	Detail
What Module 1 provides	auth.users.id as the universal user identifier. GET /api/v1/auth/me returns the authenticated user context.
What Module 2 must implement	POST /api/v1/profiles — create profile. GET /api/v1/profiles/{id} — read profile. PUT /api/v1/profiles/{id} — update.
Trigger event	auth.user.registered — Module 2 subscribes and auto-creates an empty profile for every new user
Auth dependency	Module 2 endpoints call decode_access_token() from shared kernel. No separate auth logic.
Database contract	profiles schema. profiles.user_id is a FK to auth.users.id. Never joins auth tables directly.
Zero changes to Module 1	Module 1 publishes auth.user.registered. It never calls Module 2. No imports from Module 2 in Module 1 code.

18.2 Module 3 — Orders and Payments

Contract Item	Detail
What Module 1 provides	Authenticated user identity via JWT token. Tenant context. User role for permission checks.
What Module 3 must implement	Full order lifecycle APIs. Payment gateway integration. Invoice generation. Webhook handling.
Auth dependency	@require_permission('orders:write') decorator from shared kernel RBAC. Tenant isolation via middleware.
Database contract	orders schema. orders.user_id is FK to auth.users.id. Module 3 maintains its own user snapshot for order history display.
Events published	orders.created, orders.paid, orders.cancelled, orders.refunded
Compliance note	PCI-DSS compliance boundary. When extracted to microservice, orders schema migrates to isolated database.

18.3 Module 4 — Notifications

Contract Item	Detail
What Module 1 provides	User email and phone for delivery. notification_service.py functions extracted into Module 4.
What Module 4 must implement	Template management, multi-channel delivery (email/SMS/push), delivery tracking, retry logic, user preference management.
Events subscribed	auth.user.registered (welcome email), auth.login.success (new device alert), orders.paid (receipt), orders.cancelled (notification)
Migration from Module 1	notification_service.py in Module 1 is replaced by an event publish call. Module 4 handles delivery. Zero impact on Module 1 behaviour.
User preferences	Module 4 maintains notification_preferences table. Users opt out of specific notification types.

18.4 Module 5 — Admin Dashboard

Contract Item	Detail
What Module 1 provides	RBAC role and permission system. Audit log data. User management APIs.
What Module 5 must implement	Role assignment UI, audit log viewer, system metrics dashboard, tenant management, configuration management.
Special permissions	Admin endpoints require TENANT_ADMIN or SUPER_ADMIN role. Enforced by @require_permission('admin:*) decorator.
Reporting queries	Complex aggregation queries use raw SQL via user_queries.py pattern. Read replica used for all reporting.
Events subscribed	All events — Module 5 maintains a unified activity feed for administrators.

18.5 Mobile API Layer

Contract Item	Detail
Architecture choice	Not a separate module — a dedicated API version (/api/mobile/v1) with mobile-optimised response shapes.
Auth difference	Mobile clients cannot use HttpOnly cookies. Mobile uses Bearer token in Authorization header. Refresh token stored in device secure storage (Keychain/Keystore).
Token delivery	Mobile login endpoints return tokens in response body (one-time). All subsequent requests use Authorization: Bearer header.
Push notifications	Module 4 handles push via FCM/APNs. Mobile client registers device token via POST /api/v1/notifications/device-tokens.
Offline support	Mobile endpoints include ETag and Last-Modified headers. Clients

Contract Item	Detail
	cache responses and use conditional requests.

19. Evolution Roadmap — Modular Monolith to Microservices

The evolution from Modular Monolith to Microservices is not a single big-bang migration. It is a series of small, safe extractions — each triggered by a specific business or technical condition, each completely reversible if it does not work.

19.1 The Three Evolutionary Stages

STAGE 1 — TODAY (Modular Monolith)

Single FastAPI process. All modules in one codebase.
One PostgreSQL database with schema-per-module.
Redis Streams for async events.
One Docker Compose file starts everything.

Modules: Auth (M1) + Profiles (M2) + Orders (M3)
+ Notifications (M4) + Admin (M5)

Deployment: Single VPS or small ECS cluster
Cost: \$40-\$500/month
Team size: 1-5 developers

STAGE 2 — EXTRACTION (Selective Microservices)

Extract modules that hit extraction triggers.
Most modules remain in the monolith.
Extracted services communicate via events + HTTP.

Likely first extraction: Notifications (M4)
Reason: different scale profile, different team

Likely second extraction: Orders/Payments (M3)
Reason: PCI-DSS compliance boundary

Still in monolith: Auth, Profiles, Admin

Deployment: ECS Fargate or small Kubernetes
Cost: \$500-\$2,000/month
Team size: 5-20 developers

STAGE 3 — FULL MICROSERVICES

Each module is an independent service.
Each service has its own database.
API Gateway (Kong) handles routing and auth.

Kafka handles all async communication.
Kubernetes manages all deployments.

Deployment: EKS multi-region
Cost: \$3,000-\$10,000+/month
Team size: 20+ developers, dedicated platform team

19.2 How to Extract a Module Safely

This is the step-by-step process for extracting any module from the monolith. Using Notifications (Module 4) as the example:

Step	Action	Detail
1	Verify module boundaries	Confirm Module 4 has zero direct imports from other modules. Only uses shared kernel.
2	Create new repository	notification-service/ — copy Module 4 code. Add its own Dockerfile, requirements.txt.
3	Run both in parallel	Deploy new service alongside monolith. Both subscribe to the same events. Compare outputs.
4	Shadow mode testing	New service processes events but does not send real notifications yet. Verify correctness.
5	Migrate the database schema	Move notifications schema from shared DB to dedicated DB. Run both simultaneously during transition.
6	Cut over traffic	Route all notification events to new service. Monitor for 48 hours.
7	Remove from monolith	Delete Module 4 code from monolith codebase. Monolith now publishes events only.
8	Verify	All notifications still delivered. No errors in logs. Performance improved.

Zero Downtime Guarantee

Every extraction step above is reversible. At step 6, if the new service has issues, roll back to the monolith handling events in minutes. The shadow mode parallel run in steps 3-5 means the extraction is proven correct before any production traffic is cut over.

20. Team and Process Recommendations

20.1 Team Structure per Phase

Phase	Team Size	Structure
Phase 1	1-3 developers	Full-stack developers own entire codebase. One person can run the full stack locally.
Phase 2	5-15 developers	Module teams form. Each module has 1-2 owners. Shared kernel maintained by most senior developer.
Phase 3	20+ developers	Platform engineering team owns infrastructure. Product teams own individual microservices.

20.2 Development Workflow

- Every feature developed in a feature branch — never directly on main
- Pull request requires passing CI (lint + type check + tests) before merge
- Code review required for any change to shared kernel or Module 1 auth code
- Database migrations reviewed separately — schema changes are permanent
- Deployment to staging is automatic on merge to main
- Deployment to production is manual approval after staging validation
- Hotfixes go through the same process — no direct production deployments

20.3 Module Ownership Rules

Rule	Rationale
Each module has one named owner	Clear accountability. One person approves all changes to their module.
Module 1 (Auth) has the strictest review	Security-critical. Two-person review required for any change to auth logic.
No cross-module direct imports	Enforced by architecture. CI fails if Module 2 imports from Module 3.
New events must be documented in catalogue	Event catalogue in HLD Part 2 is kept updated. New events require HLD update.
Breaking API changes require new version	v1 is never broken. New incompatible changes go to v2 with 6-month parallel period.

21. Risk Register

Architectural risks identified and mitigated at design time. Every risk has a specific mitigation built into the architecture.

#	Risk	Likelihood	Impact	Mitigation
1	Module boundaries erode over time as team grows	High	High	CI linting rules ban cross-module imports. Architecture review in every sprint.
2	Single VPS goes down in Phase 1	Medium	High	Daily automated backups to R2. Recovery procedure documented and tested.
3	PostgreSQL becomes bottleneck before Phase 2 migration	Medium	High	PgBouncer from day one. Scaling triggers defined. Vertical resize available in hours.
4	Shared kernel becomes a bloated dependency	Medium	Medium	Kernel contains only infrastructure. Business logic never enters kernel. Reviewed quarterly.
5	Event schema changes break subscribers	Medium	High	Event versioning. Old schema supported for 2 major versions. Breaking changes announced 30 days ahead.
6	Redis data loss causes OTP/rate-limit failure	Low	Medium	Redis is ephemeral by design. Loss means re-requesting OTP. Acceptable graceful degradation.
7	Third-party provider outage (Twilio/SendGrid)	Medium	Medium	Fallback providers configured. Twilio → AWS SNS. SendGrid → AWS SES. Automatic failover.
8	JWT SECRET_KEY rotation breaks all sessions	Low	High	Key rotation procedure documented. Old key accepted for 24 hours after rotation. Graceful re-login.

22. HLD Completion Summary

This three-part HLD document defines the complete architecture for the enterprise IT services platform. The following table summarises what each part covers and confirms the document set is complete.

Document	Title	Sections Covered
----------	-------	------------------

Document	Title	Sections Covered
HLD Part 1	Architecture and Topology	Executive Summary, Architecture Decision, System Topology, Module Map, Deployment Phases, API Strategy, NFRs, Compliance
HLD Part 2	Components and Data	Component Design, Data Architecture, Multi-Tenancy, Security Architecture, Event-Driven Messaging, Caching, Observability
HLD Part 3	Scale and Evolution	Scalability Strategy, Cost Model, Module Integration Contracts, Evolution Roadmap, Team Structure, Risk Register

Next Documents — LLD

The Low-Level Design documents drill into the implementation detail of Module 1 specifically. LLD Part 1 covers the complete database schema, all API endpoint contracts, and service interface definitions. LLD Part 2 covers sequence diagrams for every major flow, error handling catalogue, data flow diagrams, and the test strategy. Reply 'LLD Part 1' to continue.